

คู่มือการเขียนโปรแกรม

ไฟร์วอลล์แบบอิงบริการไดเรกทอรี

Directory Based Firewall

สำหรับโครงงานนี้ได้แบ่งการแอปพลิเคชันเป็น 2 ส่วน คือ ส่วนที่อยู่บนวินโดวส์ 2000 และส่วนที่อยู่บนลินุกซ์ ซึ่งทั้ง 2 แอปพลิเคชันจะทำงานร่วมกัน โดยแอปพลิเคชันบนฝั่งวินโดวส์ 2000 นั้นจะทำหน้าที่ในการตรวจสอบการเข้าออกของผู้ใช้ ส่วนโปรแกรมบนลินุกซ์นั้นจะทำเป็นเดมอน โดยมีหน้าที่ add/remove rules ให้กับผู้ใช้ที่เข้าหรือออกระบบ

ISAGDFW Manager For Window2000

สำหรับการเขียนโปรแกรม ISAGDFW Manager for Window2000 นั้น จะใช้ภาษา C++ ในการเขียนโปรแกรม โดยแอปพลิเคชันนี้จะแบ่งเป็น 2 ไฟล์ คือ

1. checkuserc.cpp
2. isagdfw.cpp

ไฟล์ checkuserc นั้นจะเป็นไฟล์สำหรับการตรวจสอบผู้ใช้ที่ลือคอินเข้าสู่ระบบ โดยใช้ฟังก์ชัน NetAPI ที่มีมาในระบบปฏิบัติการวินโดวส์ 2000 จากนั้นจะทำการเอาที่พุดชื่อผู้ใช้นั้นออกทาง text file ซึ่งคือ user.txt เพื่อให้ไฟล์ isagdfw อ่านค่าจากไฟล์ user.txt ไปประมวลผลต่อไป

ส่วนไฟล์ isagdfw นั้นจะเป็นไฟล์หลักของแอปพลิเคชัน โดยจะมีการเรียกใช้คำสั่งในการ execute command โดยเรียกไฟล์ checkuserc ขึ้นมาใช้งานเพื่อรับผู้ใช้ที่เข้ามาลือคอินในระบบ รวมถึงฟังก์ชันในการติดต่อกับไฟร์วอลล์ผ่านทางซ็อกเก็ตด้วย

Checkuserc.cpp

```
//header include
#ifndef UNICODE
#define UNICODE
#endif
#include <stdio.h>
#include <assert.h>
#include <fstream.h>
#include <windows.h>
#include <lm.h>
```

```

#include <process.h>

#include<wchar.h>

#include <stdarg.h>

#pragma comment(lib,"netapi32")

//end include


int wmain(int argc, wchar_t *argv[]) //main
{
    LPWKSTA_USER_INFO_0 pBuf = NULL;
    LPWKSTA_USER_INFO_0 pTmpBuf=NULL;
    DWORD dwLevel = 0;
    DWORD dwPrefMaxLen = -1;
    DWORD dwEntriesRead = 0;
    DWORD dwTotalEntries = 0;
    DWORD dwResumeHandle = 0;
    DWORD i;
    DWORD dwTotalCount = 0;
    NET_API_STATUS nStatus;
    LPTSTR pszServerName = NULL;
    FILE* stream;

    if (argc > 2)
    {
        fwprintf(stderr, L"Usage: %s [\\ServerName]\n", argv[0]);
        exit(1);
    }
    if (argc == 2)
        pszServerName = argv[1];
    do // begin do
    {
        nStatus = NetWkstaUserEnum(pszServerName, //use netapi
                                   dwLevel,
                                   (LPBYTE*)&pBuf,
                                   dwPrefMaxLen,

```

```
        &dwEntriesRead,  
        &dwTotalEntries,  
        &dwResumeHandle);
```

```
if ((nStatus == NERR_Success) || (nStatus == ERROR_MORE_DATA))
```

```
    //check for error
```

```
{
```

```
    if ((pTmpBuf = pBuf) != NULL) //check for null
```

```
{
```

```
    for (i = 0; (i < dwEntriesRead); i++)
```

```
    {
```

```
        assert(pTmpBuf != NULL);
```

```
        if (pTmpBuf == NULL)
```

```
        {
```

```
            fprintf(stderr, "An access violation has occurred\n");
```

```
            break;
```

```
        }
```

```
        pTmpBuf++;
```

```
        dwTotalCount++;
```

```
    }
```

```
    pTmpBuf--;
```

```
    stream = fopen( "user.txt", "w" );
```

```
    fprintf( stream, "%S", pTmpBuf->wkui0_username);
```

```
    fclose( stream );
```

```
}
```

```
else
```

```
{
```

```
    stream = fopen( "user.txt", "w" );
```

```
    fprintf( stream, "%S", "$");
```

```
    fclose( stream );
```

```
    exit(1);
```

```
}
```

```
}
```

```

else
{
    stream = fopen( "user.txt", "w" );
    fprintf( stream,"%S",$");
    fclose( stream );
    exit(1);
}

if (pBuf != NULL)
{
    NetApiBufferFree(pBuf);
    pBuf = NULL;
}

}

while (nStatus == ERROR_MORE_DATA); // end do
if (pBuf != NULL)
    NetApiBufferFree(pBuf);

return 0;
}

```

อธิบายโค้ด

หลังจากที่เรียกใช้ NetAPI แล้ว เราจะทำการตรวจสอบว่า การเรียกใช้ฟังก์ชันเกิดความผิดพลาดหรือไม่ โดยค่านั้นจะเก็บใน nStatus เมื่อไม่มีค่าที่ผิดพลาด รายชื่อผู้ใช้ที่ได้เคยใช้เครื่องนั้นๆ จะถูกเก็บใน pBuf ซึ่ง pBuf นั้นจะเป็น link list สำหรับเก็บชื่อผู้ใช้ แล้วนำผู้ใช้เหล่านั้นมาเก็บไว้ใน pTmpBuf ซึ่งจะ เป็น link list เช่นกัน

```

if ((nStatus == NERR_Success) || (nStatus == ERROR_MORE_DATA))
    //check for error
{
    if((pTmpBuf = pBuf) != NULL) //check for null
    {

```

```

for (i = 0; (i < dwEntriesRead); i++)
{
// สำหรับ for loop นี้ เพื่อรับค่าผู้ใช้งานตั้งแต่คนแรกจนถึงคนสุดท้าย
assert(pTmpBuf != NULL);
if (pTmpBuf == NULL)
{
fprintf(stderr, "An access violation has occurred\n");
break;
}
pTmpBuf++;
dwTotalCount++;
}

```

จากนั้นจะนำผู้ใช้คนสุดท้ายใน link list มา write stream ออกทาง text file ซึ่งผู้ใช้คนสุดท้าย ก็คือ ผู้ใช้ที่กำลังใช้งานอยู่นั่นเอง

```

pTmpBuf--;
stream = fopen( "user.txt", "w" );
fprintf( stream,"%S",pTmpBuf->wkui0_username);
fclose( stream );
}

```

ในกรณีที่ไม่มีผู้ใช้อยู่ pBuf = NULL นั้น จะทำการ write stream ออกทาง text file เป็น \$ else

```

{
stream = fopen( "user.txt", "w" );
fprintf( stream,"%S",$");
fclose( stream );
exit(1);
}
}

```

ในกรณีที่เกิดความผิดพลาดขึ้นเมื่อเรียกใช้ NetAPI จะทำการ write stream ออกทาง text file เป็น \$ เช่นกัน จากนั้นจะ return 1 ออกไป

```

else

```

```
{  
    stream = fopen( "user.txt", "w" );  
    fprintf( stream,"%S", "$");  
    fclose( stream );  
    exit(1);  
}
```

เมื่อทำการ write stream เรียบร้อยแล้ว ก็จะทำการล้าง Buffer โดยเรียกใช้ฟังก์ชัน
NetApiBufferFree(); จากนั้นจะทำการ return ค่า 0 ออกไป เพื่อจบโปรแกรม

```
if (pBuf != NULL)  
    NetApiBufferFree(pBuf);  
return 0;  
}
```

ISAGDFW Manager.cpp

```
#include <stdio.h>
#include <conio.h>
#include <process.h>
#include<fstream.h>
#include<winsock2.h>
#include<string.h>
#include<time.h>
#include"ipdef.h"

#pragma comment(lib, "Ws2_32.lib")
#pragma comment(lib,"kernel32")

#define MYPORT 1170
#define MYHOST "192.168.0.254"
//define MYHOST "192.168.2.1"

#define prog "checkuserc.exe"

typedef unsigned int SOCKET;

int sendtofw(char* msg)
{
    WSADATA wsData;
    WORD wVer = MAKEWORD(2,2);
    SOCKET minix;
    char *szName=MYHOST;
    int status;
    struct hostent *he;
    WCHAR wszRecvBuf[100];
    struct sockaddr_in their_addr;
```

```

memset(wszRecvBuf,0,sizeof(wszRecvBuf));

status = WSASStartup(wVer,&wsData);

minix=socket(AF_INET,SOCK_STREAM,0);

if (minix == INVALID_SOCKET)
{
    printf("Failed to create socket: %d\n",WSAGetLastError());
    WSACleanup();
    return -1;

}

bind(minix,(const sockaddr *)&their_addr,sizeof(their_addr));

their_addr.sin_family=AF_INET;
their_addr.sin_port=htons(MYPORT);

int len=0;

gethostname(szName,len);

he = gethostbyname(szName);

their_addr.sin_addr.S_un.S_addr = *((long *) (he->h_addr));

memset(&(their_addr.sin_zero),'\0',8);

status= connect(minix,(struct sockaddr*) &their_addr,sizeof(struct sockaddr)) ;

if(status== -1)
{
    printf("\nfail to connect");
    WSAGetLastError();
    WSACleanup();
    return -1;
}

char* msg2 = (char*) malloc (sizeof(char*));

status = send(minix,(char*)msg,100,0);

printf("\n\n ***** Send Action %s: ",msg);

printf(" to Firewall *****\n");

```



```

        printf("\n=====
==");
        if(status<0)
        {
            printf("\nfail to send");
            WSAGetLastError();
            WSACleanup();
            return status;
        }

        closesocket(minix);
        WSACleanup();
        return 1;
    }

```

```

int readfile(char* user)
{
    strcpy(user,"$");
    ifstream in("user.txt",ios::in);
    in>>user;
    if(strcmp(user,"$")) return 1;
    return 0;
}

```

```

void checkuser(char* user,char* usertmp,char* msg,char* ipp,char* ip[],int i)
{
    if (readfile(user))
    {
        if((strcmp(ip[i],user)!=0) && ip[i]!="$") //check for change user
        {
            /*          if(user=="$")
            {

```

```

printf("\n=====
=====");

printf("\n\n ***** ");
printf("%s",ip[i]);
printf(" Log off ");
printf("At IP : %s",ipp);
printf(" *****");

//printf("\nUser log off :%s",ip[i]);
strcpy(msg,ip[i]);
strcat(msg,":");
strcat(msg,ipp);
strcat(msg,"D");
sendtofw(msg);
strcpy(usertmp,"$");
ip[i]=usertmp;
}

else
{*/

printf("\n=====
=====");

printf("\n\n ***** ");
printf("%s",ip[i]);
printf(" Log off ");
printf("At IP : %s",ipp);
printf(" *****");

/*printf("\n***** User log off :%s ",ip[i]);
printf(" *****");*/

//printf("\nUser log off :%s",ip[i]);
strcpy(msg,ip[i]);

```

```

        strcat(msg,":");
        strcat(msg,ipp);
        strcat(msg,"D");
        sendtofw(msg);
        usertmp=(char*) malloc (sizeof(char*));
        strcpy(usertmp,user);
        ip[i]=usertmp;
        printf("\n=====
=====");

        printf("\n\n ***** ");
        printf("%s",ip[i]);
        printf(" Log in ");
        printf("At IP : %s",ipp);
        printf(" *****");

        strcpy(msg,ip[i]);
        strcat(msg,":");
        strcat(msg,ipp);
        strcat(msg,"A");
        sendtofw(msg);

    //}
}

else if(ip[i]=="$") //check for

login

{

    /*printf("\n***** User log in:%s ",user);
    printf(" *****");
    printf("\n***** %s",user);
    printf(" Log in at IP %S",ip[i]);*/
    //printf("\nUser log in:%s",user);
    usertmp=(char*) malloc (sizeof(char*));
    strcpy(usertmp,user);
    ip[i]=usertmp;

```

```
printf("\n=====");

    printf("\n\n ***** ");
    printf("%s",ip[i]);
    printf(" Log in ");
    printf(" At IP : %s",ipp);
    printf(" *****");

    strcpy(msg,user);
    strcat(msg,":");
    strcat(msg,ipp);
    strcat(msg,"A");
    sendtofw(msg);

}

else

//check for log off

{

    if(strcmp(ip[i],"$"))
    {

        printf("\n=====");

        printf("\n\n ***** ");
        printf("%s",ip[i]);
        printf(" Log off ");
        printf(" At IP : %s",ipp);
        printf(" *****");


        //printf("\nUser log off:%s",ip[i]);
        strcpy(msg,ip[i]);
        strcat(msg,":");
        strcat(msg,ipp);
        strcat(msg,"D");
        sendtofw(msg);
        strcpy(ip[i],"$");
```

```

    }
}
}

```

```

void main()

```

```

{

```

```

    char* ip[254];

```

```

    for (int i=0;i<50;i++) {

```

```

        ip[i]=(char*) malloc (sizeof(char*));

```

```

        ip[i]="$";

```

```

    }

```

```

    char* usertmp=(char*) malloc (sizeof(char*));

```

```

    char* user=(char*) malloc (sizeof(char*));

```

```

    char* msg=(char*) malloc(sizeof(char*));

```

```

//    printf("\nDetected User....");

```

```

    printf("\n----- Directory Service based Firewall -----");

```

```

    printf("\n----- ISAGDFW Manager for Window2000 -----\\n\\n");

```

```

while(1)

```

```

{

```

```

    for(int i=0;i<2;i++)

```

```

    {

```

```

        switch(i)

```

```

        {

```

```

            case 0 :

```

```

            {

```

```

                if(_spawnl(_P_WAIT ,prog,prog ,"client1",NULL )==0)

```

```

                    checkuser(user,usertmp,msg,IP1,ip,i);

```

```

                else

```

```

                    checkuser(user,usertmp,msg,IP1,ip,i);

```

```

            }break;

```

```

        case 1:{
            if(!_spawnl(_P_WAIT ,prog, prog,"client2",NULL )==0)
                checkuser(user,usertmp,msg,IP2,ip,i);
            else
                checkuser(user,usertmp,msg,IP2,ip,i);
            }break;

        case 2 :
            {
                if(!_spawnl(_P_WAIT ,prog, prog,"client3",NULL )==0)
                    *
                    *      ๒๒๓๓๓๓ case 251
                    *

                checkuser(user,usertmp,msg,IP3,ip,i);
            else
                checkuser(user,usertmp,msg,IP3,ip,i);
            }break;
        case 252 :
            {
                if(!_spawnl(_P_WAIT ,prog, prog,"client253",NULL )==0)
                    checkuser(user,usertmp,msg,IP253,ip,i);
            }break;

        default:break;
    }
}

//      printf("\nDetected User....");
      Sleep(10000);

}
}

```

อธิบายโค้ด

ในฟังก์ชัน sendtofw นั้นจะเป็นการติดต่อซ็อกเก็ตเพื่อส่ง message ไปยังไฟร์วอลล์ โดย message นี้จะทำการ concat ชื่อผู้ใช้งาน ไอพีแอดเดรส และตามด้วย แอคชั่น ซึ่ง จะมี A คือ ให้ add rules และ D คือ ให้ Remove Rules เช่น userA:192.168.2.99A โดยส่วนนี้จะเป็นการเขียนโปรแกรมผ่านทางซ็อกเก็ต ซึ่งได้อธิบายไว้ในปฏิญานพนธ์ แล้ว

สำหรับฟังก์ชัน readfile นั้นจะทำหน้าที่ read file จาก user.txt นำค่ามาเก็บ ซึ่งจะเป็นผู้ใช้ที่ใช้งานอยู่บนเครื่องไอพีที่ตรวจสอบ

สำหรับฟังก์ชัน checkuser นั้น จะทำการตรวจสอบผู้ใช้ เนื่องจากว่าการทำงานของโปรแกรมเป็นการทำงานวนลูปเข้าไปเรื่อยๆ ดังนั้น ในการวนแต่ละรอบ จะได้ชื่อผู้ใช้ทุกกรอบ ฟังก์ชันนี้จะตรวจสอบว่าเป็นผู้ใช้คนเดิมหรือไม่ หรือมีการเปลี่ยนแปลงผู้ใช้หรือไม่ จากนั้นจะทำการ concat ผู้ใช้กับไอพีและเพิ่มแอคชั่นที่ได้กล่าวมาแล้วไปใน message แล้วส่งให้ฟังก์ชัน sendtofw เพื่อทำการติดต่อส่งค่าไปยังไฟร์วอลล์ต่อไป

ใน main นั้นจะทำการ initialize ชื่อผู้ใช้ในแต่ละ ip โดยให้มีค่าเป็น \$ ซึ่งจะหมายถึงไม่มีผู้ใช้ใช้งานอยู่ โดยตัวแปร ip นั้นจะหมายถึงชื่อผู้ใช้ในแต่ละ ip โดยจะเป็น array of char* ขนาด 254

จากนั้นในส่วน while loop นั้นจะทำการ execute file โดยเรียกใช้ไฟล์ checkuserc.exe ขึ้นมาโดยจะมี argument เป็น ip ไปทุกๆ ip วนอย่างนี้ไปเรื่อยๆ ในแต่ละ case แล้วจะทำการเรียกใช้ฟังก์ชัน checkuser ขึ้นมาเพื่อทำงานต่อไป

ISAGDFW For Linux

โปรแกรม isagdfw จะประกอบด้วย file ที่เป็น .c อยู่ทั้งหมด 5 files คือ

exec.c

main.c

server.c

timestamp.c

word.c

ซึ่งจะเป็นการแยกเอา function ที่มีการทำงานที่คล้ายคลึงกันออกเป็น file ต่าง โดยมี Makefile เป็น file ที่ทำการบอกการทำงาน make ว่าต้องมีการอ้างเอา header file(.h) และใช้ object ต่างๆ จากที่ได้ รวมถึงเป็นการระบุว่าใน file .c ต่างๆนั้นมีการ compile ออกเป็น object ใดบ้างเพื่อให้ linker ทำการ link object ต่างๆที่ต้องใช้งานใน execute file isagdfw ได้อย่างถูกต้อง ดังตัวอย่างของ Makefile

```
OBJECTS= main.o server.o exec.o word.o
```

```
all: main.o server.o exec.o word.o
```

```
    g++ $(CFLAG) -o isagdfw main.o server.o exec.o word.o
```

```
main.o: main.c word.o server.o exec.o word.h exec.h server.h
```

```
    gcc $(CFLAG) -c -ggdb main.c
```

```
server.o: server.c server.h
```

```
    gcc $(CFLAG) -c -ggdb server.c
```

```
exec.o: exec.c exec.h
```

```
    gcc $(CFLAG) -c -ggdb exec.c
```

```
word.o: word.c word.h
```

```
    gcc $(CFLAG) -c -ggdb word.c
```


clean:

```
rm -rf *.o isagdfw
```

รูปที่ 1 Makefile

จากรูปจะเห็นได้ว่า Makefile ได้แยกออกเป็น สามส่วน คือ ส่วนที่จะทำการประกาศว่า object ต่างๆที่ต้องการทำการ compile มีอะไรบ้าง, ส่วนที่ประกาศว่า แต่ละ object จะทำการ compile หรือ link ด้วย source file และ object ไດ ส่วนสุดท้ายนั้นเป็นการประกาศถึงส่วนที่ทำการ clean เวลาเราต้องการให้ make ทำการ ลบ file ต่างๆเพื่อการเริ่มการ compile ใหม่

```
#include <stdio.h>
```

```
#include <sys/types.h>
```

```
void parse(char *line, char **argv)
```

```
{
    while (*line != '\0') {    /* if not the end of line ..... */
        while (*line == ' ' || *line == '\t' || *line == '\n')
            *line++ = '\0';    /* replace white spaces with 0 */
        *argv++ = line;        /* save the argument position */
        while (*line != '\0' && *line != ' ' &&
            *line != '\t' && *line != '\n')
            line++;            /* skip the argument until ... */
        *argv = '\0';          /* mark the end of argument list */
    }
}
```

```
void execute(char **argv)
```

```
{
    pid_t pid;
    int status;

    if ((pid = fork()) < 0) {    /* fork a child process */
```

```

printf("*** ERROR: forking child process failed\n");
exit(1);
}
else if (pid == 0) { /* for the child process: */
    if (execvp(*argv, argv) < 0) { /* execute the command */
        printf("*** ERROR: exec failed\n");
        exit(1);
    }
}
else { /* for the parent: */
    while (wait(&status) != pid); /* wait for completion */
}
}
}

```

รูปที่ 2 *exec.c*

client.c นั้นจะประกอบด้วย 2 function เพื่อใช้ในการ execute program โดย function parse นั้นจะทำการตัดเอา command ที่ได้มาซึ่งอยู่ในรูปของ string บรรทัดเดียว ออกเป็น argument ต่างๆ โดยแยกเอา argument ออกเป็น string ย่อยๆ ซึ่งอยู่ใน array โดยที่ array filed สุดท้าย เป็น NULL เพื่อนำไป execute ใน function execute โดยใน function execute นั้นจะทำการรับเอา argument ที่ได้จาก function parse มาทำการ execute โดยทำการ fork process ออกมาแล้วทำการ execute program นั้นๆ ผ่าน function execvp() โดย execvp() นั้นจะต้องการ full path เป็น argument ในการทำงาน

```

void parse(char *, char **);
void execute(char **);

```

รูปที่ 3 *exec.h*

exec.h นั้นจะเป็นการประกาศ header ของ file exec.c โดยจะประกาศถึงว่า function ที่อยู่ใน exec.c นั้นมี function ใด มี argument ใด และ return ค่าแบบใด

```

#include<stdio.h>
#include<fcntl.h>

```

```

#include<stdlib.h>
#include<unistd.h>
#include<errno.h>
#include<string.h>
#include<sys/stat.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<sys/wait.h>
#include<signal.h>
#include<time.h>
#include "exec.h"
#include "word.h"
#include "server.h"
#include "timestamp.c"

#define MYPORT 1170          // define port
#define MAXDATASIZE 100     // define datasize
#define BACKLOG 10          // define log
#define FILENAME "rules.txt" // define rule file
#define LOG "isagdfw.log"   // define log file

int main(int argc,char* argv[])
{
    int i;// new counter
    int sockfd,new_fd;
    struct sockaddr_in my_addr;
    struct sockaddr_in their_addr;
    int sin_size;
    struct sigaction sa;
    int yes=1;
    int display=0;          // 0 if in deamon mode 1 if in display mode
    FILE* log; // file pointer

```

```

if(argc>1)    //check mode of process
    if(strcmp(argv[1],"-display")==0) display=1;

if(display==0)
{
i=fork();
if(i<0) exit(0); // exit if error
if(i>0) exit(0); // exit if parent daemon

setsid();    // new process group
for (i=getdtablesize();i>=0;--i) close(i); // close all descriptors
i=open("/dev/null",O_RDWR); dup(i); dup(i); // handle standart I/O
umask(007); //set permission
chdir("/tmp/isagdfw"); //change dir
};

if((sockfd=socket(AF_INET,SOCK_STREAM,0))==-1)// create socket to listen
{
    perror("socket");
    exit(1);
}

if(setsockopt(sockfd,SOL_SOCKET,SO_REUSEADDR,&yes,sizeof(int))==-1)// set
socket option
{
    perror("setsockopt");
    exit(1);
}

my_addr.sin_family = AF_INET;           // set address family
my_addr.sin_port = htons(MYPORT); // set port
my_addr.sin_addr.s_addr=INADDR_ANY;    //set in address
memset(&(my_addr.sin_zero),'\0',8);    // pading

```

```

if (bind(sockfd, (struct sockaddr *)&my_addr, sizeof(struct sockaddr))== -1) // bind socket
{
    perror("bind");
    exit(1);
}

if (listen(sockfd, BACKLOG) == -1)
{
    perror("listen");
    exit(1);
} // listen

sa.sa_handler = sigchld_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART;
if (sigaction(SIGCHLD, &sa, NULL) == -1)
{
    perror("sigaction");
    exit(1);
}

while(1) // main accept() loop
{
    sin_size = sizeof(struct sockaddr_in);
    if ((new_fd = accept(sockfd, (struct sockaddr *)&their_addr, &sin_size)) == -1)
    {
        perror("accept");
        continue;
    } // new_fd is socket for except connection in new process
    if (display == 1)
    {
        timestamp(1);
        printf("\nServer : Got Action from \t->
%s\n", inet_ntoa(their_addr.sin_addr));
    }
}

```

```

    }
else
{
    timestamp(0);
    log=fopen(LOG,"a");
    fprintf(log, "\nServer : Got Action from \t-> %s \t\ttime:
",inet_ntoa(their_addr.sin_addr));
    fclose(log);
};
if(!fork())
{
    char rule4parse[200];
    char *argv[20];
    char *user=(char*)malloc(sizeof(char*));
    char *addr=(char*)malloc(sizeof(char*));
    char *rule=(char*)malloc(sizeof(char*));
    char *fromfile;
    char addremove;
    char *tempPtr;
    char buf[MAXDATASIZE];
    int i,len,j;
    size_t* filelen;

    close(sockfd);
    if(recv(new_fd, buf, MAXDATASIZE-1, 0)==-1)perror("recv");
    if(display==1)
        printf("Server : Got String \t\t-> %s\n",buf);
    else
    {
        log=fopen(LOG,"a");
        fprintf(log,"Server : Got String \t\t-> %s\n",buf);
        fclose(log);
    };
};

```

```

splitaddr(buf,addr,user);// spleit address and user

tempptr=addr;           //
len=strlen(addr)-1;      //
for(i=0;i<len;i++)       //
    tempptr++;           // get command ... add or remove
if(*tempptr=='A')addremove='A'; //
else addremove='D';      //
*tempptr='\0';           //

if(display==1)
    printf("\nuser  : %s \t\t address : %s \t command:
%c\n",user,addr,addremove);
else
{
    log=fopen(LOG,"a");
    fprintf(log,"\nuser  : %s \t\t address : %s \t command:
%c\n",user,addr,addremove);
    fclose(log);
};

// get rules
fromfile=read_file(filelen);
rule=cut_rule(fromfile,user);
if(display==1)
    printf("user  : %s \t\t rules  : %s\n-----
=====
\n",user,rule);
else
{
    log=fopen(LOG,"a");
    fprintf(log,"user  : %s \t\t rules  : %s\n",user,rule);
    fprintf(log,"-----
=====
\n");
    fclose(log);
};

```

```

//add-remove rules
strcpy(rule4parse,rule);
parse(rule4parse,argv);          // parse rule for excute
//check add-remove and set ipaddress
for(j=0;j<20;j++)
{
    if(0==strcmp(argv[j],"command")) //
    {
        //
        if(addremove=='A')argv[j]="-A"; // set command
        if(addremove=='D')argv[j]="-D"; //
    };
    //

    if(0==strcmp(argv[j],"sourceipaddress"))//
    {
        //
        strcpy(argv[j],addr); // set ipaddress
        break; //
    };
    //
};

execute(argv); // excute command

close(new_fd); // close socket
exit(0);

}

close(new_fd);
}

return 0;

}

```

รูปที่ 4 main.c

main.c นั้นจะเป็น file ที่มี source code ส่วนที่จะทำการทำงานหลักของ program และเป็น ส่วนที่ทำงานในการเริ่มต้นของ program นอกจากส่วนที่ทำการ define ค่าต่างๆที่ใช้งาน และการ include

header ที่ใช้งานทั้งหมดแล้ว จะเริ่มจากการ check mode การทำงานว่าทำงานใน mode ไດ หากทำงานในโหมด display ก็จะมีค่าเพื่อตรวจสอบในส่วนอื่น แต่หากเป็นการทำงานใน daemon mode นั้นก็จะทำการ fork process ใหม่ขึ้นมาทำงานต่อ และ set group กับค่าต่างๆ ก่อนทำการ exit parent process ออกไป เพื่อให้ process สามารถทำงานใน background mode ได้อย่างสมบูรณ์และไม่เป็น zombie process จากนั้นเริ่มทำการ สร้าง socket ขึ้นมา และ set ค่า parameter ต่างๆ ให้เรียบร้อย ก่อนที่จะทำการ bind socket นั้นๆ เพื่อให้สามารถใช้งานได้ จากนั้นทำการวนลูป listen() เพื่อรอรับ connection ที่จะเข้ามา เมื่อมี connection เข้ามาก็จะเริ่มการทำงานด้วยการ fork process ขึ้นมารับหน้าที่ในการทำงานต่อ แยก ข้อมูลที่ได้รับมาออกด้วย function splitaddr() ซึ่งจะมีข้อมูล user, address และคำสั่งว่าจะทำการลบ หรือเพิ่มกฎ จากนั้นจะนำเอา user name มาทำการ lookup ใน text file ผ่าน function cut_rule() เพื่อทำการแยกเอา rule เฉพาะที่เป็นของ user นั้นๆ แล้วนำไปผ่าน function parse() เพื่อจัดให้ได้ array of argument พร้อมทั้งทำการ execute() ต่อไป เมื่อทำการ execute() เรียบร้อยแล้วจะทำการ exit() ออกจาก child process นั้นๆ

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<errno.h>
#include<string.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<sys/wait.h>
#include<signal.h>

void sigchld_handler(int s)
{
    while(wait(NULL)>0);
}

void splitaddr(char *in,char *addr,char *user)
{
    int len,i;
    char *temp;
    len=strlen(in);
    for(i=0;i<len;i++)
```

```

{
    if(*in==':')
    {
        in++;
        temp=in;
        len=strlen(in);
        for(i=0;i<len;i++)
        {
            if(*temp=='\r'){*temp='\0';break;}
            else{temp++;};
        }
        strcpy(addr,in);
        break;
    }
    else
    {
        strncat(user,in,1);
        in++;
    }
};
}

```

รูปที่ 5 server.c

server.c นั้นจะเป็นส่วนที่ทำการ แยก user กับ address ออกมาจากข้อมูลที่ได้รับมา โดยเป็นการวนลูปเพื่อทำการหา “:” และทำการ copy string ออกมาเพื่อ return ค่ากลับไป

```

void sigchld_handler(int );
void splitaddr(char *,char *,char *);

```

รูปที่ 6 server.h

server.h นั้นจะเป็นการประกาศ header ของ file server.c โดยจะประกาศถึงว่า function ที่อยู่ใน server.c นั้นมี function ไດ มี argument ไດ และ return ค่าแบบใด

```

#include<stdio.h>
#include<fcntl.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<sys/stat.h>
#include<signal.h>
#include<time.h>

#define LOG "isagdfw.log"

void timestamp(int i)
{
    if(!fork())
    {
        struct timeval tv;
        struct tm* ptm;
        char time[40];
        long milliseconds;
        FILE* fd;

        gettimeofday(&tv,NULL);
        ptm=localtime(&tv.tv_sec);

        strftime(time,sizeof(time),"%Y-%m-%d %H:%M:%S",ptm);
        milliseconds=tv.tv_usec/1000;
        if(i==1)
            printf("%s.%03ld\n",time,milliseconds);
        else
        {
            fd=fopen(LOG,"a");
            fprintf(fd,"%s.%03ld\n",time,milliseconds);
            fclose(fd);
        }
    };
}

```

```

        exit(0);

    };

}

```

รูปที่ 7 *timestamp.c*

timestamp.c นั้นจะเป็น function ที่ทำการ fork process ขึ้นมาทำการเขียน timestamp ลงไป
ใน text file

```

#include<stdio.h>
#include<unistd.h>
#include<string.h>
#include<fcntl.h>
#include<sys/stat.h>
#include<sys/types.h>
#include<sys/time.h>
#include<time.h>

#define FILENAME "rules.txt"

char* read_file(size_t* length)// read text file and return stream of char
{
    size_t len;
    int fd;
    struct stat file_info;
    char* buffer;

    fd=open(FILENAME,O_RDONLY);
    fstat(fd,&file_info);
    len=file_info.st_size;
    if(!S_ISREG(file_info.st_mode)){
        close(fd);
        return NULL;
    }
}

```

```

    }

    buffer=(char*)malloc(len);

    read(fd,buffer,len);

    close(fd);

    return buffer;
}

char* cut_rule(char* input,const char* keyword)// for cut rules from stream of file and return
only rule for that user

```

```

{
    char* tempptr;                                //temp pointer
    char* rules=(char*)malloc(sizeof(char)); //to store rules to return
    char* name=(char*)malloc(sizeof(char)); //to store name for compare
    int len=strlen(input);                        //length of string
    int i;                                        //AUX count
    tempptr=input;
    for(i=0;i<len;i++)
    {
        if(*tempptr==':')
        {
            if(0==strcmp(name,keyword))
            {
                tempptr++;
                strcpy(rules,tempptr);
                break;
            }else
            {
                while(*tempptr!='\n')
                {
                    i++;
                    tempptr++;
                };
                name=NULL;
                name=(char*)malloc(sizeof(char));
                tempptr++;
            }
        }
    }
}

```

```

        };

    } else
    {
        strncat(name,tempptr,1);
        tempptr++;
    };
}

len=strlen(rules);
tempptr=rules;
for(i=0;i<len;i++)
{
    if(*tempptr=='\n')
    {
        *tempptr='\0';
        break;
    };
    tempptr++;
};
return rules;
}

```

```

char add_remove(char *input)
{
    int len,i;
    char *tempptr;
    char result;
    tempptr=input;
    len=strlen(input)-1;
    for(i=0;i<len;i++)
        tempptr++;
    if(*tempptr=='A')result='A';
    else result='D';
}

```

```

*tempPtr='\0';

return result;

};

```

รูปที่ 8 word.c

word.c นั้นจะเป็นส่วนที่สร้างขึ้นมาเพื่อทำการเกี่ยวกับการอ่าน file โดย function read_file() นั้นจะทำการอ่าน file ขึ้นมาทั้งหมด โดยจะอยู่ในรูปของ pointer to character โดยจะเป็น string ทั้ง file แล้ว return string ทั้งหมดกลับออกไป ส่วน function cut_rule() นั้นจะทำการรับ input และ keyword ที่จะทำการตัดออกมา แล้วทำการวนลูป check keyword ที่ต้องการแล้วทำการ return ค่าของ string เฉพาะบรรทัดที่ขึ้นต้นด้วย keyword กลับออกไป function สุดท้ายนั้นคือ function add_remove() เพื่อทำการตรวจสอบว่า ข้อมูลที่รับมานั้นมีคำสั่งในการ add หรือ remove แล้วทำการ return ค่ากลับออกไปหากเป็นการ add จะ return character 'A' และ 'D' ในกรณีที่เป็นคำสั่ง remove

```

char* read_file(size_t* );// read text file and return stream of char

char* cut_rule(char* ,const char* );// for cut rules from stream of file and return only rule for
that user

char add_remove(char *);

```

รูปที่ 9 word.h

word.h นั้นจะเป็นการประกาศ header ของ file word.c โดยจะประกาศถึงว่า function ที่อยู่ใน word.c นั้นมี function ไດ มี argument ไດ และ return ค่าแบบใด

ติดต่อสอบถาม

นายรัฐพล ปฏิการมณฑล

rattp@hotmail.com

นายสมศักดิ์ วงศ์สินสกุล

ched_ped@hotmail.com